

Database

PostgreSQL 16, one database per deployment, migrated with Alembic. Everything runs inside containers — there is no `psql` on the host.

Opening a shell

```
make shell-db          # psql inside the db container
```

Or use the containerised GUI that ships with the dev stack:

```
make dev                # then open http://localhost:8081
# System: PostgreSQL · Server: db · User/Password/Database from .env
```

Ad-hoc query without an interactive shell:

```
docker compose exec -T db psql -U finmanager -d finmanager \
  -c "SELECT ownership_status, count(*) FROM lego_set_instances GROUP BY
1;"
```

Conventions

Rule	Why
UUID v7 primary keys, generated in Python (<code>app/core/ids.py</code>)	Time-sortable without a sequence, safe to expose
NUMERIC(10,2) for every <code>_eur</code> column	Money is decimal; floats and minor-unit integers are both banned
is_deleted + deleted_at soft delete	Financial history is voided, never destroyed
created_at / updated_at with <code>timestampz</code>	Store UTC; business dates keep their local calendar date
Evolving public enums as VARCHAR + CHECK	Adding a value is a one-line migration, not a type rewrite
Naming convention on every constraint (<code>app/models/base.py</code>)	Alembic autogenerate produces stable, reviewable diffs

Schema map

Shared core (`app/models/core.py`) — brief §1a

Table	Purpose
<code>merchants</code>	Global reference data (<code>RETAIL/BANK/INSURER/...</code>), validated Portuguese NIF
<code>categories</code>	Hierarchy with <code>parent.domain == child.domain</code> and <code>level==1 ↔ parent_id IS NULL</code> ; grocery is 3-tier and seeded from the pt-PT taxonomy
<code>tags</code>	Household-scoped cross-cutting labels
<code>documents</code>	Content-addressed attachments; <code>storage_path</code> is outside any web root
<code>links</code>	Polymorphic reconciliation edges — no real FKs; compensated by <code>(from_type, from_id) / (to_type, to_id)</code> indexes and an app-level type allow-list
<code>review_tasks</code>	Backs the shared Review Queue
<code>audit_logs</code>	Every financial mutation and lifecycle change; the sole source of historical state
<code>settings</code>	Confidence thresholds and provider opt-ins, scoped <code>GLOBAL/HOUSEHOLD/ENTITY/MODULE</code>
<code>import_batches</code> , <code>processing_jobs</code>	Ingestion bookkeeping; <code>processing_jobs.idempotency_key</code> is unique so retries never double-apply

Household (M7, `app/models/household.py`)

`households · users · household_members · entities · sessions` — five tables, three roles, no per-entity read isolation. `users.is_dependent = true` implies `password_hash IS NULL`, enforced by a `CHECK`.

LEGO (M9, `app/models/lego.py`)

Three tables, deliberately:

- `lego_set_models` — catalog identity plus the single hand-maintained `current_value_eur / value_updated_at`. A **partial unique index** enforces one set number per entity while the row is alive and has a number, so soft-deleted rows and MOCs do not collide.
- `lego_set_instances` — one row per physical copy; there is no `quantity`. `acquisition_transaction_id` is a plain UUID column, not an FK — see [ADR 0005](#).
- `lego_storage_locations` — flat `area + container`, unique per entity, `capacity_pct` constrained to 0–100.

There is **no** valuation-history table, no image table and no external-listing table. Value history is recoverable from `audit_logs`; images reuse `documents`; marketplace links are built client-side from a template.

Migrations

```
make revision M="add lego wishlist"    # autogenerate against the live
schema
make migrate                          # upgrade head
make downgrade                        # roll back one revision
```

Migrations are **additive and reversible**. Every one has a real `downgrade()`. `docker-entrypoint.sh` runs `alembic upgrade head` before the API starts, so a fresh `docker compose up` always lands on a migrated schema.

Autogenerate needs the models imported — `alembic/env.py` imports `app.models`, which re-exports every model, so a new table only needs to be added to `app/models/__init__.py`.

Performance notes

The indexes that matter today:

- `ix_lego_set_instances_entity_status (entity_id, ownership_status, is_deleted)` — the shape of every collection listing and KPI aggregate.
- `ix_lego_set_instances_model_id` — copy grouping and `owned_copies_count`.
- `ix_audit_logs_record (table_name, record_id, created_at)` — reconstructing an object's history.

When the ledger arrives, validate with `EXPLAIN (ANALYZE, BUFFERS)` against a production-scale seed rather than assuming; the NFR is <800 ms p95 over ten years of data.